Основы Объектно-ориентированного программирования

Лекция Шаблоны, линейные списки

Шаблоны

- Позволяют параметризировать определение функции или класса так, чтобы это определение могло применяться к аргументам (данным) разных типов
- Результат – можно не дублировать один и тот же код для разных типов

Пример определения шаблона – поиск максимума

```
// См. пример "max"
// Шаблон функции поиска максимума
// Здесь T обозначает какой-либо тип
template <class T> const T& getMax(
    const T& a, const T& b)
{
    return a>b?a:b;
}
// Каковы требования к типу T?
```

Пример использования шаблона

```
// С указанием специализации
int main(void)
{
    int a=2, b=3;
    cout<<"Max(a,b)="<<getMax<int>(a,b)<<endl;
    double x=1.6e+3, y=1.58e+3;
    cout<<"Max(x,y)="<<getMax<double>(x,y)<<endl;
    Rational m(3,4), n(6,7);
    cout<<"Max(m,n)="<<getMax<Rational>(m,n)<<endl;
    return 0;
}
```

Пример использования шаблона

```
// Без указания специализации
int main(void)
{
    int a=2, b=3;
    cout<<"Max(a,b)="<<getMax(a,b)<<endl;
    double x=1.6e+3, y=1.58e+3;
    cout<<"Max(x,y)="<<getMax(x,y)<<endl;
    Rational m(3,4), n(6,7);
    cout<<"Max(m,n)="<<getMax(m,n)<<endl;
    return 0;
}
```

Компиляция шаблонов

- Компилятор автоматически генерирует определения функций (классов) для всех требуемых типов
- Для этого он должен иметь доступ к **определениям** функций (объявлений недостаточно)
- По этой причине как объявления, так и определения функций размещаются в **заголовочном** файле

Пример шаблона – линейный список

- Элементы списка могут располагаться в памяти не подряд
- Место под элементы выделяется и освобождается по необходимости
- Каждый элемент, помимо полезных данных, хранит указатель(и) на соседний элемент(ы)
- Достоинства, недостатки?

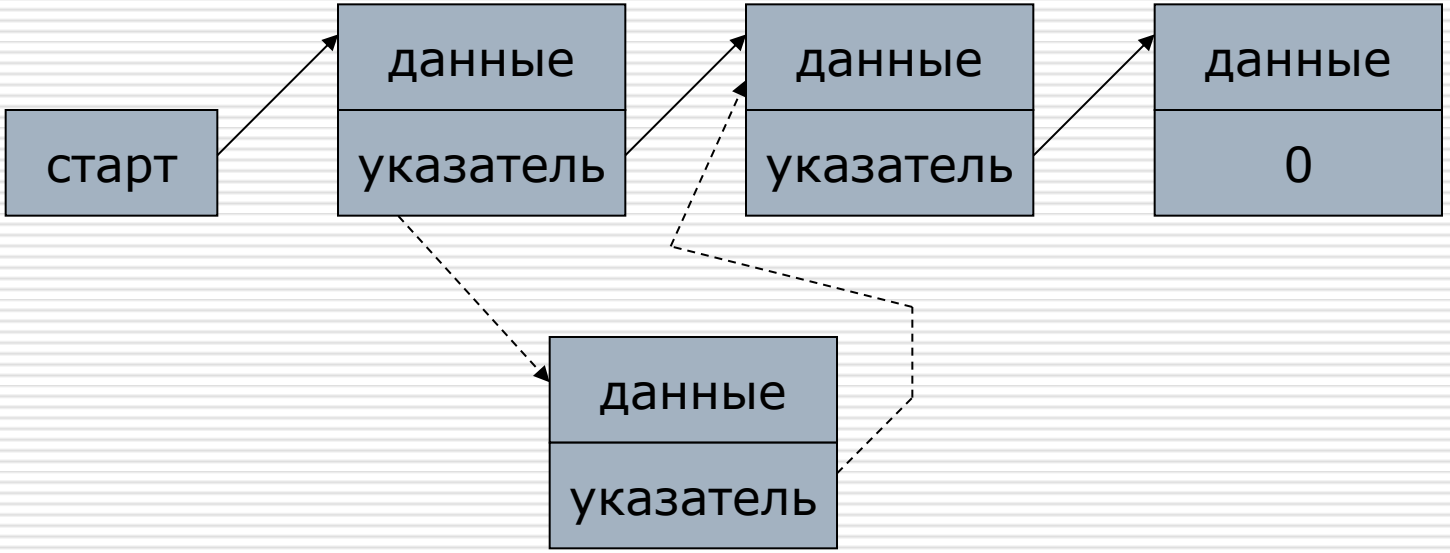
Линейный список – достоинства и недостатки

- Достоинства
  - высокая скорость операций вставки и удаления, O(1) (если знать куда)
  - возможность создавать списки любого размера
- Недостатки
  - низкая скорость операции доступа по индексу, O(N)
  - низкая скорость поиска элементов, O(N)

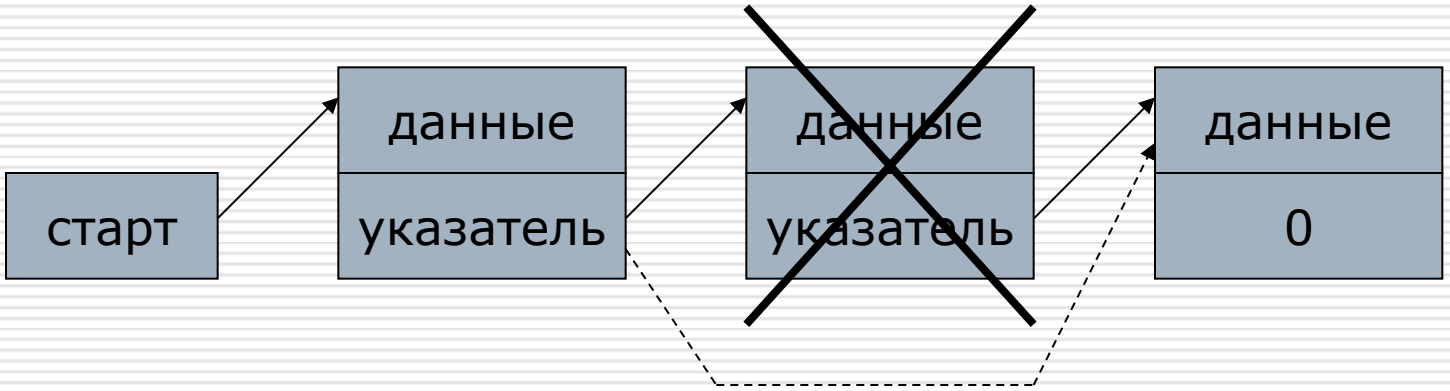
Виды линейных списков



Вставка в линейный список



Удаление из линейного списка



Реализация однонаправленного линейного списка на C++

- См. пример LineList
- Требуются следующие классы:
  - Элемент линейного списка (LineListElem)
  - Линейный список (LineList)

Шаблон для LineListItem

```
template <class T> class LineListElem
{
   T data;
   LineListElem* next;
public:
   LineListElem(const T& adata,
      LineListElem* anext);
   const T& getData() const;
   LineListElem* getNext();
   // В объявлениях друзей
   // template повторяется
   template <class T> friend class LineList;
};
```

Шаблон для LineList

```
template <class T> class LineList {
   LineListElem<T>* start; // Начало списка
   LineList(const LineList& list); // Зачем?
   LineList& operator =(const LineList& list);
public:
   LineList();
   ~LineList();
   LineListElem<T>* getStart();
   void deleteFirst();
   void deleteAfter(LineListElem<T>* ptr);
   void insertFirst(const T& data);
   void insertAfter(LineListElem<T>* ptr, const T& data);
   template <class T> friend ostream& operator <<(
      ostream& out, LineList& list);
}; // Требования к T?
```

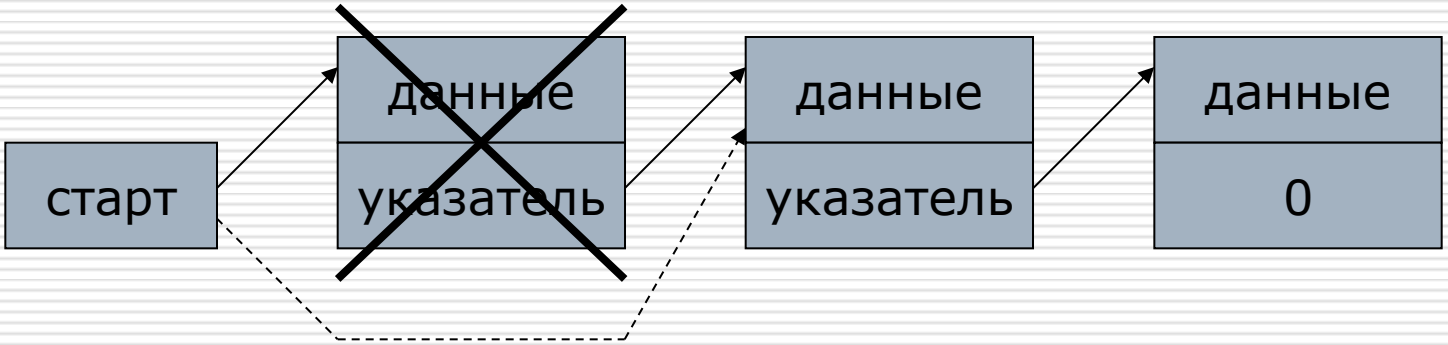
Методы LineListItem – определения

```
template <class T> LineListElem<T>::LineListElem(
    const T& adata, LineListElem<T>* anext)
{
    data = adata;
    next = anext;
}

template <class T>
    LineListElem<T>* LineListElem<T>::getNext()
{
    return next;
}
```

Удаление первого элемента

```
template <class T> void LineList<T>::deleteFirst()
{
   if (start)
   {
      LineListElem<T>* temp = start->next;
      delete start;
      start = temp;
   } else throw LineListException();
}
```



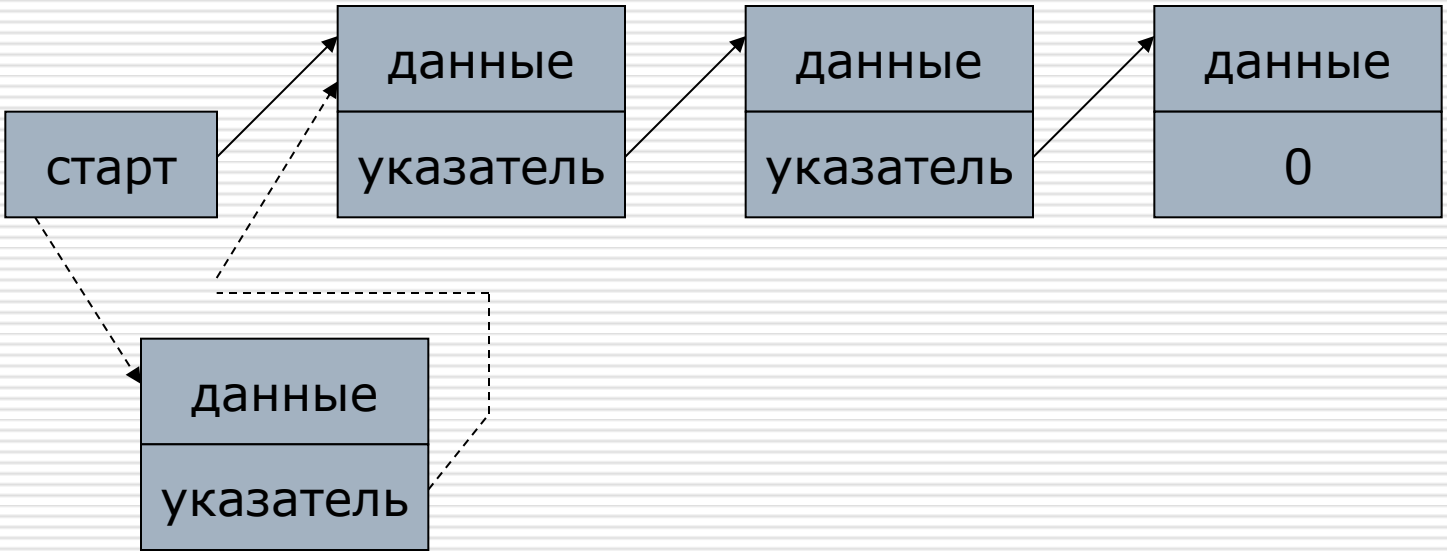
Конструктор и деструктор

```
template <class T> LineList<T>::LineList()
{
   start = 0;
}

template <class T> LineList<T>::~LineList()
{
   while (start)
      deleteFirst();
}
```

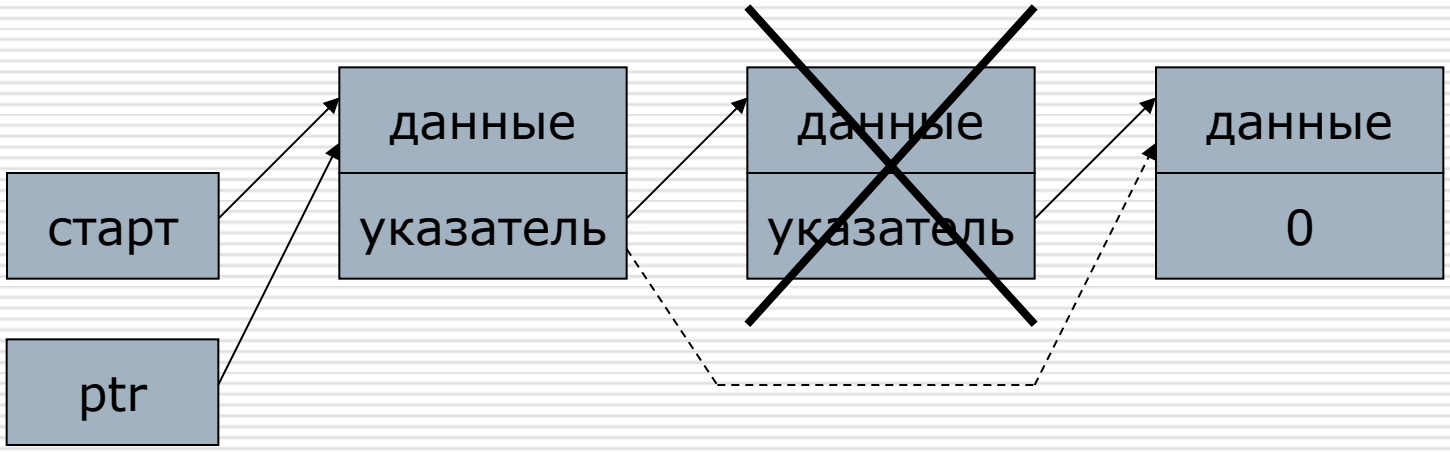
Вставка первого элемента

```
template <class T> void LineList<T>::insertFirst(
    const T& data)
{
    LineListElem<T>* second = start;
    start = new LineListElem<T>(data, second);
}
```



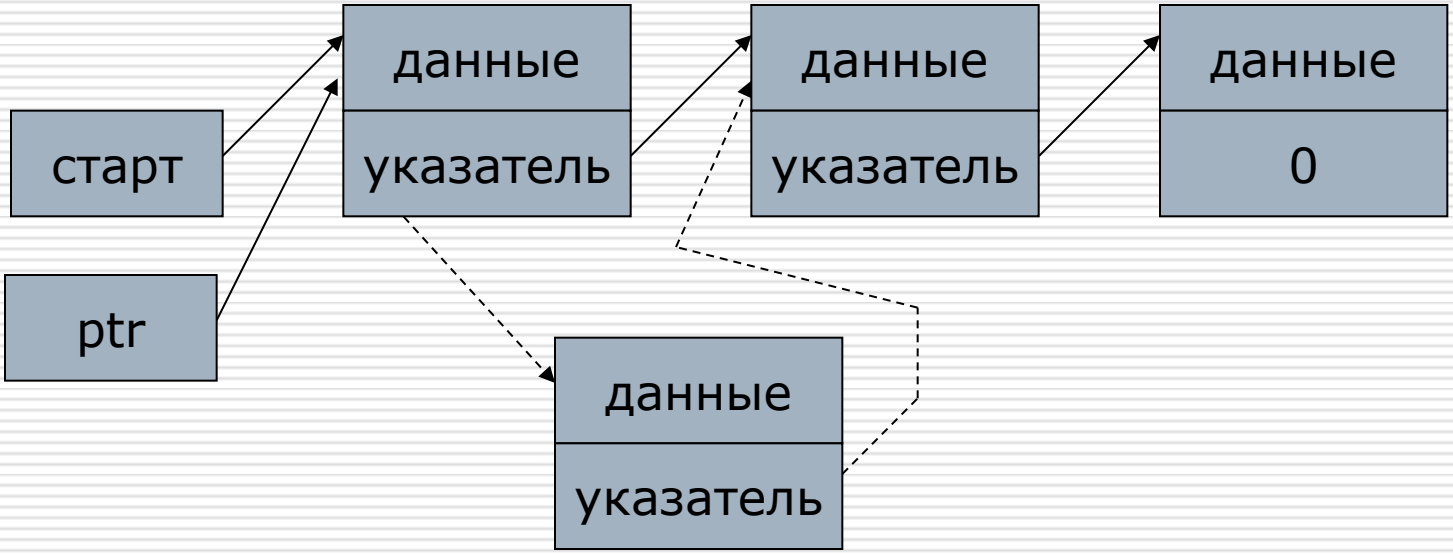
Удаление после данного элемента

```
template <class T> void LineList<T>::deleteAfter(
        LineListElem<T>* ptr) {
   if (ptr && ptr->next) {
      LineListElem<T>* temp = ptr->next;
      ptr->next = ptr->next->next;
      delete temp;
   } else throw LineListException();
}
```



Вставка после данного элемента

```
template <class T> void LineList<T>::insertAfter(
        LineListElem<T>* ptr, const T& data) {
    if (ptr) {
        LineListElem<T>* temp = ptr->next;
        ptr->next = new LineListElem<T>(data, temp);
    }
}
```



Оператор вывода

```
template <class T> ostream& operator <<
      (ostream& out, LineList<T>& list)
{
   LineListElem<T>* ptr = list.start;
   if (!ptr)
      out<<"EMPTY ";
   else while (ptr)
   {
      out<<ptr->getData()<<' ';
      ptr = ptr->getNext();
   }
   return out;
}
```

Использование шаблона класса

- В отличие от обычного класса, **заголовочный файл** должен содержать как **определение класса**, так и **определения его функций**

Использование шаблона класса

```
int main(void)
{
   setlocale(LC_ALL, "Russian");
   LineList<int> list; // Указываем, список какого типа
   cout<<"Начало: "<<list<<endl;
   list.insertFirst(10);
   LineListElem<int>* ptr = list.getStart();
   list.insertAfter(ptr, 15);
   list.insertAfter(ptr->getNext(), 12);
   list.insertFirst(7);
   cout<<"Шаг 1: "<<list<<endl;
   // …
   return 0;
}
```

Достоинства и недостатки однонаправленного списка

- Достоинства
  - быстро вставляет и удаляет **после**
  - быстро вставляет и удаляет **в начале**
- Недостатки
  - медленно вставляет и удаляет **перед**
  - медленно вставляет и удаляет **в конце**
- Как справиться с недостатками?

Задача

- Мы не реализовали операции обращения к элементам по индексу
- Предположим, нам необходимо с какой-либо целью перебрать все элементы (например, найти сумму всех элементов списка) – как это лучше сделать, пользуясь **готовыми** шаблонами LineListElem и LineList?

Hint: основные методы шаблонов LineListElem, LineList

```
template <class T> class LineListElem {
public:
   LineListElem(const T& adata, LineListElem* anext);
   const T& getData() const;
   LineListElem* getNext();
};
template <class T> class LineList {
public:
   LineList();
   ~LineList();
   LineListElem<T>* getStart();
   void deleteFirst();
   void deleteAfter(LineListElem<T>* ptr);
   void insertFirst(const T& data);
   void insertAfter(LineListElem<T>* ptr, const T& data);
};
```

Решение

```
LineListElem<int> list;
// ...
int sum = 0;
// Используем переменную-итератор
for (LineListElem<int>* it=list.getStart();
                        it; it=it->getNext())
    sum += it->getData();
cout<<"Сумма = "<<sum<<endl;
```

Аргументы шаблона

- Язык C++ допускает шаблоны с несколькими аргументами
  - **template** <**class** KEY, **class** VAL> …
- В качестве аргументов могут использоваться не только типы, но и значения:
  - **template** <**class** T, **int** SIZE> …

Стандартная библиотека шаблонов

- Одной из важнейших составляющих языка C++ является стандартная библиотека шаблонов (standard template library, STL)
- STL содержит большое количество готовых шаблонов, которые могут быть использованы при написании программ

Использование готовых шаблонов из библиотеки STL

- Все шаблоны STL определены в пространстве имен std
  - **using namespace** std;
- Как правило, использование шаблона требует подключения одноименного заголовочного файла
  - **#include** <vector>
  - **#include** <list>

Готовые шаблоны контейнеров

- Основные
  - vector – массив переменного размера
  - list – список
  - deque – двухсторонняя очередь
- Дополнительные
  - queue – односторонняя очередь
  - stack – стек
- Специальные
  - basic_string – строка
- Ассоциативные
  - map – таблица ключ-значение на основе бинарного дерева
  - set – множество

Требования к элементам контейнера

- открытый конструктор копирования
- открытый оператор присваивания
- открытый деструктор
- отдельные операции также требуют
  - открытый конструктор по умолчанию
  - открытый оператор сравнения на равенство
  - открытый оператор сравнения <

Шаблон <list>

- Внутренняя организация – на основе двунаправленного линейного списка
- Элементы добавляются по мере необходимости
- Похож на LineList – но двунаправленный

Основные свойства списка

- size() – количество элементов в списке на данный момент
  - может быть изменено вызовом resize()
- max_size() - максимально возможное количество элементов в списке

Основные возможности списка

- Конструкторы
  - по умолчанию (пустой список)
  - заданного размера
  - заданного размера и наполнения
  - копирования
- Оператор присваивания
- Операторы сравнения
- *Обращение по индексу*
  - для списка данная функция **не поддерживается**

Основные возможности списка

- Очистка (clear)
- Проверка на пустоту (empty)
- Добавление последнего элемента (push_back) или первого элемента (push_front)
  - размер увеличивается на 1
- Удаление последнего элемента (pop_back) или первого элемента (pop_front)
  - размер уменьшается на 1

Основные возможности

- изменение порядка элементов на обратный (reverse)
- сортировка (sort)
- слияние двух списков с упорядочением (merge)
- удаление дублирующихся элементов (unique)

Итераторы

- Специальные объекты, предназначенные для перебора элементов контейнера; по принципу действия напоминают указатели
- Для списка – фактически, единственное средство доступа
- Определяются как:

```
list<int>::iterator it; // или
vector<float>::iterator it2;
```

Возможности итераторов

- Разадресация (*it)
- Перевод на следующий элемент (it++)
- Перевод на предыдущий элемент (it--)
- Сложение, вычитание (it+2, it-3, it2-it1, только для вектора)

Перебор элементов списка с помощью итератора

```
list<int> mylist;
// …
int sum = 0;
for (list<int>::iterator it=mylist.begin();
        it!=mylist.end(); it++)
{
   cout<<*it<<' ';
   sum += *it;
}
cout<<endl<<sum<<endl;
```

Возможности вектора и списка, связанные с итераторами

- begin() – получение итератора, указывающего на 1-й элемент
- end() – получение итератора, указывающего **за** последний элемент
- insert(it, value) – вставка **перед** it элемента value
- insert(it, itfrom, itto) – вставка **перед** it элементов [itfrom, itto)
- assign(itfrom, itto) – очистка с последующим заполнением элементами [itfrom, itto)

Достоинства и недостатки списка

- Достоинства
  - Быстрые операции со случайными элементами (вставка, удаление)
- Недостатки
  - Отсутствие доступа по индексу
  - Медленный поиск

Шаблон <deque>

- Так называемый **дек** или двухсторонняя очередь
- Внутренняя организация – на основе кольцевого массива или нескольких кольцевых массивов

Основные свойства очереди

- size() – количество элементов в списке на данный момент
  - может быть изменено вызовом resize()
- max_size() – максимально возможное количество элементов в списке

Основные возможности очереди

- Конструкторы
  - по умолчанию (пустая очередь)
  - заданного размера
  - заданного размера и наполнения
  - копирования
- Оператор присваивания
- Операторы сравнения
- Обращение по индексу [i] или at(i)
  - диапазон значений 0…size()-1
  - в at при неверном индексе происходит ошибка при выполнении программы

Основные возможности очереди

- Очистка (clear)
- Проверка на пустоту (empty)
- Добавление последнего элемента (push_back) или первого элемента (push_front)
  - размер увеличивается на 1
- Удаление последнего элемента (pop_back) или первого элемента (pop_front)
  - размер уменьшается на 1
- Основные операции с итераторами (begin, end, insert, assign)

Достоинства и недостатки очереди

- Достоинства
  - Быстрые операции с первым и последним элементом (вставка, удаление): O(1)
  - Быстрый доступ по индексу (хотя и уступающий вектору): O(1)
- Недостатки
  - Медленные операции с внутренними элементами (вставка, удаление): O(N)

Вспомогательные контейнеры - stack, queue

- Реализуются на основе одного из основных контейнеров, обычно на основе *deque*
- Основные возможности
  - конструкторы
  - проверка на пустоту empty()
  - получение размера size()
  - добавление элемента push()
  - удаление элемента pop()
  - получение верхнего элемента top() / front()

Множество – set<T>

- Хранит каждое значение один раз
- Внутри – разновидность бинарного дерева поиска (подробнее позже)
- Трудоёмкость основных операций O(log(N))
  - insert(val)
  - erase(val)
  - it=find(val)
- Также простые операции с O(1)
  - begin(), end()
  - clear(), empty(), size()

Строки STL

- Строка, состоящая из 8-битных символов (char), использующих одну из 8-битных кодировок, например, *CP-1251*

```
basic_string<char> // или просто
string
```

- Строка, состоящая из 16-битных символов (wchar_t), использующих кодировку *Unicode*

```
basic_string<wchar_t> // или просто
wstring
```

- Требуют подключения <string>

Основные возможности строк STL

- Конструкторы
  - по умолчанию
  - из низкоуровневой строки (char*)
  - копирования
- Операторы
  - присваивания =, assign
  - добавления (конкатенации) +=, append
  - индексации []
  - сравнения
  - ввода-вывода >> << getline
- Получение размера – length, size
- Получение низкоуровневой строки – c_str()

Основные возможности строк STL

- Вставка строки insert(index, string)
- Получение подстроки substr(begin, length)
- Поиск подстроки find(string)
- Сравнение compare(string)
- Работа с итераторами

Пример работы со строками – сортировка файла по алфавиту

- См. пример **Alphabet**
- Задача разбивается на три:
  - чтение строк из файла
  - сортировка строк
  - запись строк в файл
- Для хранения строк применим список – в этом случае функция сортировки будет готовой

Чтение строк из файла

```
list<string>* readFile(const char* filename)
{
   ifstream in(filename);
   if (!in.is_open())
      return 0;
   list<string>* result = new list<string>();
   string str;
   getline(in, str);
   while (!in.fail())
   {
      result->push_back(str);
      getline(in, str);
   }
   return result;
}
```

Запись строк в файл

```
bool writeFile(list<string>* pList,
               const char* filename)
{
   ofstream out(filename);
   if (!out.is_open())
      return false;
   for (list<string>::iterator it=pList->begin();
        it!=pList->end(); it++)
      out<<*it<<endl;
   return true;
}
```

Главная функция

```
int main(void)
{
   list<string>* pList = readFile("in.txt");
   if (!pList) {
      cout<<"File in.txt does not exists"<<endl;
      return -1;
   }
   pList->sort();
   if (!writeFile(pList, "out.txt")) {
      cout<<"Cannot open output file"<<endl;
      return -2;
   }
   cout<<"Success!"<<endl;
   return 0;
}
```

Итоги

- Рассмотрены способы создания шаблонов на языке C++
- Рассмотрены готовые шаблоны контейнеров: вектор, список, очередь, стек, строка
- Далее: графовые алгоритмы